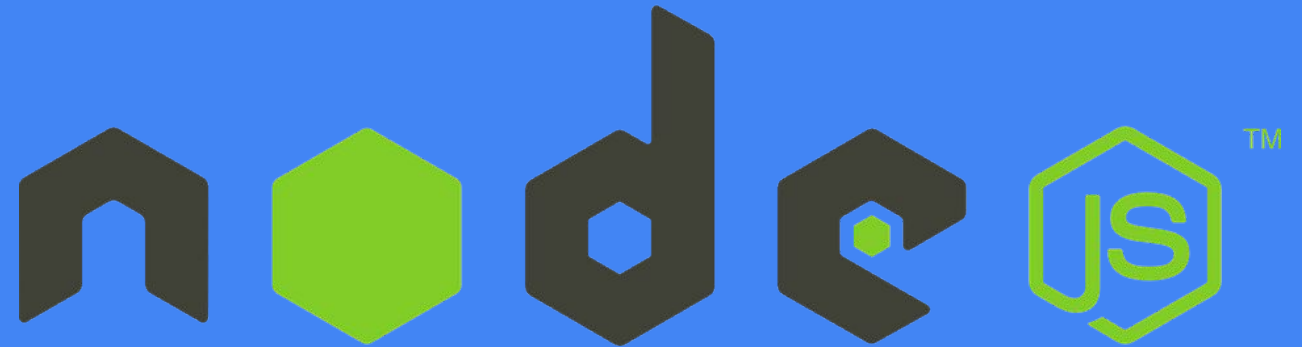




express

Node.js + Express

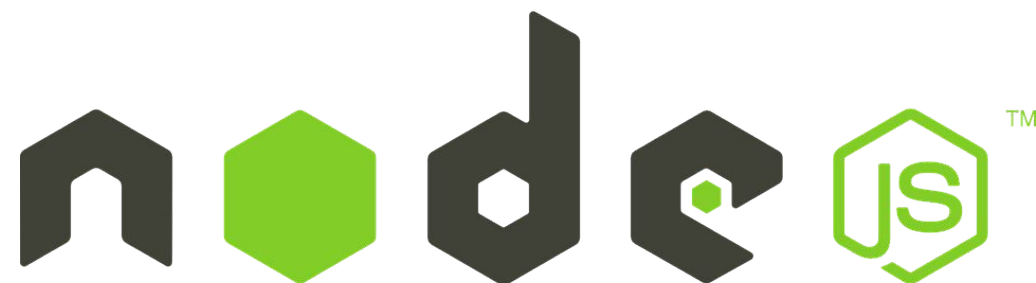
1. Introdução ao Node.js

O que é Node.js

O [Node.js](https://nodejs.org) é um sistema open source em tempo execução (*runtime*)

JavaScript, criado com o mecanismo JavaScript [V8](https://v8.dev) utilizado pelo Google Chrome.

O V8, implementa o *ECMAScript* (especificação JavaScript) e é escrito em C++, e roda em Windows, Mac e Linux, permitindo a execução de programas escritos com JavaScript.



Documentação oficial: nodejs.org

O Node.js é um sistema open source, de código aberto, mantido pela [OpenJS Foundation](https://openjsfoundation.org).

Para quê serve o Node.js?

- Criar servidores web
- Criar APIs
- Manipular arquivos e persistir localmente
- Alto fluxo de input/output de dados via rede
- Middleware
- IoT
- Streaming
- Como é possível instalar o Node em sistemas operacionais muito reduzidos, a versatilidade faz do Node útil para ser utilizado em diversas ocasiões.
- Há versões do Node para Window, MacOS, Linux e outros SO.

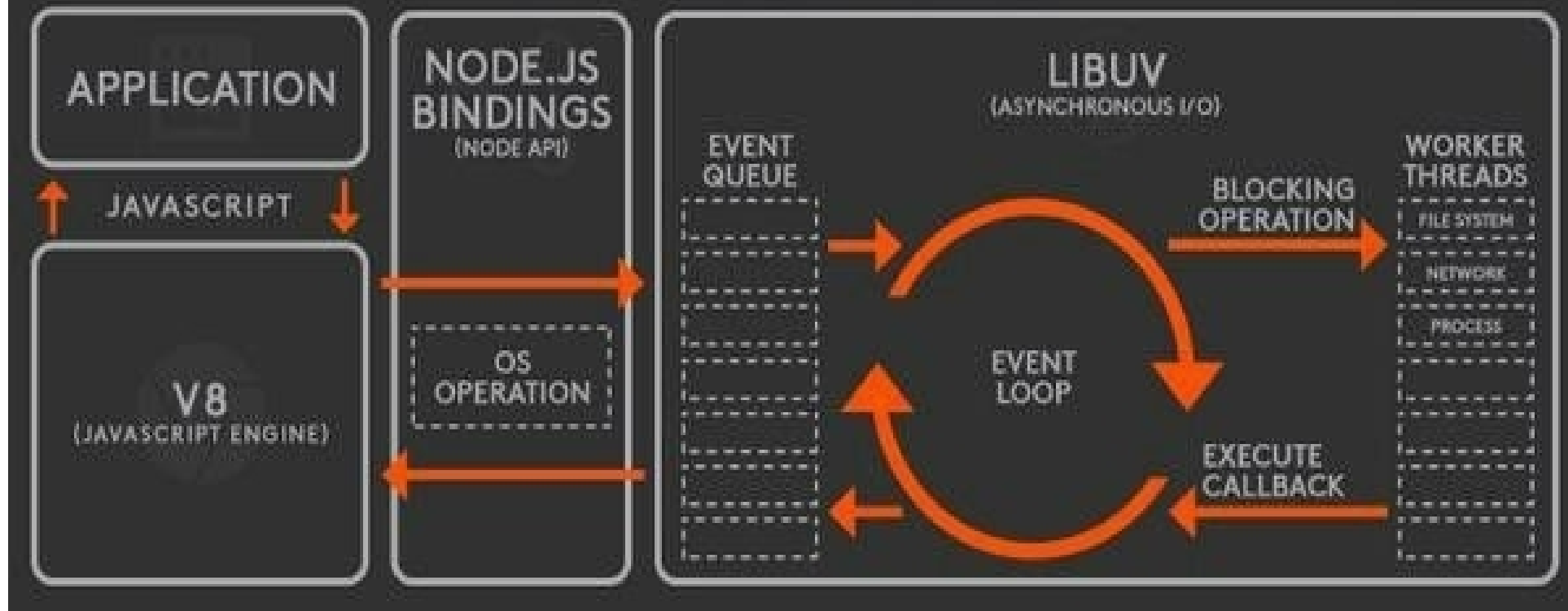
Quando usar o Node.js?

- MVP, POC
- Projetos web complexos com front-end
- Server Side Rendering
- Microsserviços
- Conhecimento já adquirido de JavaScript para aplicações cliente
- Open Source
- Baixo custo
- Módulos em JavaScript
- Fácil Integração

Como o Node.js funciona?

THE NODE.JS SYSTEM

A THIRDCENTY-FOUR
MODULES



Características do Node.js

- Chrome **V8 Engine** - C/C++ (node-gyp compila plugins)
- `require("module_name")` - Usa **CommonJS**
<http://nodebr.com/como-funciona-a-funcao-require-do-node-js/>
<https://codesandbox.io/s/node-require-module-export-import-v6zww>
- Vem com **built-in Objects**: *path, http, fs, url*
- Exemplo mínimo **servidor web** Node: <https://codesandbox.io/s/smallest-node-server-gwhl5>
- Suporta Módulos: *module.export*
- Repositório Node Package Manager (**NPM**)
- Long Term Version (**LTS**)
- Node Version Manager (**NVM**)
- Consultar features: <https://node.green/>

Objectos Globais, Eventos e Requests

- Objectos Globais

- *global* (variável global, namespaces: **fs**, **http**, **url**, **path**, **https**, **stream**, **v8**, **Array**, **Buffer**, **ArrayBuffer**)
- *process* (**I/O**, eventos, **pid**, **stdin**, **stdout**, **stderr**, **exit**, **kill**) (*EventEmitter*)

- Eventos

- JS é single-thread
- loop events, callbacks, **EventEmitter**
- libev, libeio
- **emit()**
- **setTimeout()**, **setInterval()**

- Request

- **http.get()**, **request**, **node-fetch**, **async/await**, **Promise**, **r2**, **axios**

Como criar um
servidor web
com Node.js?

Instalando o Node.js

Existem várias maneiras de instalar o Node.js. A versão atual, com suporte a longo tempo (**LTS** — *Long Term Support*) é a mais recomendada, mas cada projeto pode exigir uma versão específica do Node.js. Ou seja, se você instalar apenas uma versão na sua máquina, poderá ser complicado mudar caso um projeto precise de uma versão específica, mais antiga, por exemplo. Sendo assim, existem ao menos duas opções de instalar o Node.js:

- **Opção 1 — Baixando arquivo Binário**
- **Opção 2 — Utilizando o NVM**

Node Package Manager (NPM)

- O Node.js já vem com o **NPM** instalado. Ele é um Gerenciador de Pacotes (*Package Manager*) e serve para instalar módulos JavaScript em ambiente de desenvolvimento ou para serem usados na sua aplicação em produção.
- Como instalar módulos como NPM?
 - `$ npm install request -g` // instala globalmente
 - `$ npm install express --save` // dependência da app
 - `$ npm install node-sass --save-dev` // dependência apenas para desenvolvimento da app
- Para buscar todos os pacotes disponíveis no NPM: <https://www.npmjs.com/>
- Uma alternativa ao NPM é o **Yarn**, desenvolvido pelo Facebook Labs: <https://yarnpkg.com/en/>

Criando um novo projeto para NPM

Passo 1

Node.

Exemplo package.json

```
{
  "name": "my_package",
  "description": "",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "https://github.com/username/my\_package.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "bugs": {
    "url": "https://github.com/username/my\_package/issues"
  },
  "homepage": "https://github.com/username/my\_package"
}
```

2.

ExpressJS

Um framework Node.js

Express é um **framework**

do seu

Adicionar Express como Dependência

Passo 2

Confira no
item

VSCode

O arquivo

Passo 3

Módulos Node

Na pasta

módulos

Hello World com ExpressJS

app.js

```
const express = require('express')
const app = express()
const port = 3000

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(`Example app listening at http://localhost:\${port}`)
})
```

Hello World com ExpressJS

constante

porta onde vai **rodar** nossa

rota e o

send

porta

log

Hello World com ExpressJS

Para **executar**
`node app.js`

log

get

Hello World.

Criar um servidor web

Passo

arquivo **app.js**

```
var express = require('express');  
var app = express();
```

```
app.get('/', function (req, res) {  
  res.send('Hello World!');  
});
```

```
app.listen(3000, function () {  
  console.log('Example app listening on port 3000!');  
});
```

Exercicio...!

```
app.post("/api", () => {})
```

3.

Microserviços

Microserviços, APIs e REST

No livro "[Martin Fowler](#)" define
conjuntos

APIs

softwares

e devolvem

REST

GET,

POST, PUT, DELETE

interface

RESTful

arquitetura

protocolo

CRUD

CRUD

A sigla **CRUD**

método

Update

API?

Express

para "

HTTP. O "C" de **Create**

Read

DELETE

métodos do **HTTP** para fazer **CRUD**. APIs

(Create)

método

cUrl

via **POST**

um

Middlewares

_____ são **módulos**

alterar do **Express**

json

converter **formato**

listas utilizando o **objeto** [Array](#)

inicializar Arrays **literais**

variáveis

método "**push**

res.json

JSON

JSON

Notation)

em **JavaScript**.

Objetos literais

chaves

vírgulas

objeto. É possível

listas

Use os

e informe o **índice**

recuperar

a partir do **zero**.

API Node REST

Como persistir dados no servidor web?

```
const express = require('express')
const app = express()
const port = 3000

let items = []
app.use(express.json())

app.get('/', (req, res) => {
  res.json(items[1])
})

app.post('/', (req, res) => {
  items.push(req.body.name)
  console.log(req.body)
  res.send(`Criando dados de ${req.body.name}`)
})

app.listen(port, () => {
  console.log(`Example app listening at http://localhost:3000`)
})
```

Bibliografia

BÁSICA

- CROCKFORD, Douglas. O Melhor do JavaScript, Rio de Janeiro: Alta Books, 2008.
- EIS, Diego. Guia Front-End - O caminho das pedras para ser um dev Front-End, São Paulo: Casa do Código, 2015.
- POWERS, Shelley. Aprendendo Node, São Paulo: Novatec, 2017.
- MAZZA, Lucas. HTML5 e CSS3 - Domine a web do futuro, São Paulo: Casa do Código, 2012.
- LOPES, Sérgio. A Web Mobile - Design Responsivo e além para uma Web adaptada ao mundo mobile, São Paulo: Casa do Código, 2015.
- MORRISON, Michael. Use a Cabeça JavaScript, Rio de Janeiro: Alta Books, 2008.
- MORAES, William Bruno. Construindo Aplicações com NodeJS, São Paulo: Novatec, 2018.
- ALMEIDA, Flávio. Cangaceiro JavaScript - Uma aventura no sertão da programação, São Paulo: Casa do Código, 2017.

COMPLEMENTAR

- FLANAGAN, David. JavaScript: O Guia Definitivo, Porto Alegre: Bookman, 2013.
- HAVERBEKE, Marijn. Eloquent JavaScript, São Francisco: No Starch Press, 2018.
- SILVA, Maurício Samy. CSS Grid Layout: Criando layouts CSS profissionais, São Paulo: Novatec, 2019.
- OSMANI, Addy. Learning Javascript Design Patterns, Sebastopol: O'reilly, 2012.
- STEFANOV, Stoyan. Padrões JavaScript, São Paulo: Novatec, 2010.
- ZAKAS, Nicholas C. JavaScript de Alto Desempenho, São Paulo: Novatec, 2010.
- RESIG, John. Segredos do Ninja Javascript, São Paulo: Novatec, 2013.
- GRONER, Loiane. Estruturas de dados e algoritmos com JavaScript: Escreva um código JavaScript complexo e eficaz usando a mais recente ECMAScript, São Paulo: Novatec, 2019.
- ROCHA, Zeno. 14 Hábitos de Desenvolvedores Altamente Produtivos, [S.l.]: Amazon Kindle, 2020. Disponível em <<http://amazon.com/>>